

Universidad de Cundinamarca

Facultad de Ingeniería

Optimización puntual de la planificación académica de la Facultad de Ingeniería mediante algoritmos de grafos

Henry Ricaurte Mora - 20221020084

Felipe Cardenas Mora - 20232010145

Nicolas Avendaño Barajas - 20231020113

Proyecto implementado en **TypeScript** para asignación de salones y
generación de horarios

25 de mayo de 2026

Índice general

1. Introducción	4
2. Resumen ejecutivo	5
3. Contexto general del repositorio	6
4. Algoritmos y estructuras de apoyo	7
4.1. Propósito	7
4.2. Estructuras fundamentales	7
4.2.1. <code>Vertex<T></code>	7
4.2.2. Listado de restricciones (resumen)	8
4.2.3. Diagrama de clases (restricciones)	9
4.2.4. Uso de imágenes (rutas y URLs)	9
4.2.5. Implementación: verificador (Allocator)	9
4.2.6. Implementación: verificador (Student)	10
4.2.7. Ejemplos concretos	11
4.2.8. Buenas prácticas y organización	11
4.2.9. Ejemplo concreto: preferencia de laboratorio por la tarde	12
4.3. Constraints para horarios estudiantiles	12
4.3.1. Ejemplo concreto: no solapamiento	13
5. Desglose de src/universidad-proyecto	14
5.1. Resumen funcional	14
5.2. Punto de entrada	14
5.3. Componentes clave	14
5.3.1. Cargador de datos	14
5.3.2. <code>ConflictGraphBuilder</code>	15

5.3.3. ScheduleSolver	15
5.3.4. ScheduleService	15
5.3.5. StudentScheduleSolver	15
5.4. Rutas y controladores principales	16
5.5. Consideraciones de diseño	16
6. Especificación técnica	17
6.1. Problema seleccionado	17
6.2. Cómo se modela en el código	17
6.3. Algoritmos seleccionados	18
6.4. Entrada y salida	18
6.5. Validación básica	18
7. API y endpoints	19
7.1. Visión general	19
7.2. GET /schedule	19
7.3. GET /schedule/grafos	20
7.4. POST /student/schedule	20
7.5. GET /student/schedule/:estudianteId	21
7.6. GET /student/schedules	21
7.7. Formato de respuesta	22
8. Modelado del problema	23
8.1. Descripción del conflicto	23
8.2. Interpretación del grafo	23
8.3. Reglas que aplica el código	23
8.4. Resultado esperado	23
9. Algoritmos empleados	24
9.1. Ruta principal del proyecto	24
9.2. Coloreo global	24
9.3. Planificación estudiantil	24
9.4. Otras heurísticas presentes en el repositorio	24
10. Ejecución de algoritmos	25

10.1. Construcción del horario global	25
10.2. Uso de constraints durante la asignación	25
10.3. Planificación estudiantil	25
10.4. Recuperación de datos ya calculados	26
11. Interfaz de usuario y visualización	27
11.1. Arquitectura del frontend	27
11.2. Placeholders para capturas	27
11.3. Ejemplo de hook (resumen)	27
11.4. Flujo de interacción	28
12. Conclusiones	29
13. Anexos	30
13.1. Estructura del repositorio	30
13.2. Datasets	30
13.3. Imágenes	30
13.4. Comandos útiles	30

1. Introducción

Este trabajo aborda la optimización puntual de la planificación académica de la Facultad de Ingeniería mediante técnicas de grafos implementadas en **TypeScript**. El problema tratado es la asignación de salones y franjas horarias a bloques académicos evitando conflictos entre estudiantes, profesores y tipos de espacio.

La solución se organiza en capas: carga y normalización de datos, construcción del grafo de conflictos, algoritmos de resolución (coloreo y caminos mínimos), generación del horario final, exposición por API y visualización en frontend. Todas las decisiones de diseño responden a criterios de reproducibilidad y trazabilidad del código.

Objetivo específico: producir una asignación válida de salones para los bloques académicos del dataset, comparar al menos tres heurísticas de coloreo y generar horarios personalizados por estudiante usando técnicas de caminos mínimos cuando sea necesario.

2. Resumen ejecutivo

Se presenta una solución técnica para asignación de salones y generación de horarios en la Facultad de Ingeniería basada en grafos. El enfoque combina:

- construcción de un grafo de conflictos a partir de datos institucionales;
- aplicación de heurísticas de coloreo (D-Satur, Welsh-Powell, voraz) para asignar salones;
- uso de Dijkstra y estrategias de respaldo para generar horarios estudiantiles que minimicen fatiga y cambios de sede;
- exposición de resultados mediante API REST y visualización en una SPA React.

Resultados claves: la pipeline permite comparar estrategias, validar soluciones y producir artefactos reproducibles (salidas JSON y métricas por escenario). Se proponen tres escenarios de evaluación: dos basados en datasets incluidos y uno simulado de mayor densidad conflictiva.

3. Contexto general del repositorio

El código se organiza en tres bloques funcionales:

- `src/algoritmos`: implementaciones de algoritmos de grafos reutilizables (coloreo, caminos mínimos, flujo, MST, recorridos).
- `src/estructuras`: tipos y estructuras de soporte (`Vertex`, `Edge`, `Graph`, proyecciones, `PriorityQueue`).
- `src/universidad-proyecto`: solución aplicada con cargadores de datos, constructor de grafo de conflictos, solucionadores, API y frontend.

La separación permite reutilizar algoritmos y estructuras en otros problemas y facilita la comprobación experimental de distintas heurísticas sobre la misma instancia.

4. Algoritmos y estructuras de apoyo

4.1 Propósito

Proveer una base estable y tipada para implementar algoritmos de grafos y construir el grafo de conflictos. El uso de **TypeScript** facilita genéricos, interfaces y trazabilidad de tipos.

4.2 Estructuras fundamentales

4.2.1 Vertex<T>

```
1 export class Vertex<T> {
2   constructor(public readonly id: string, private value: T, private color?:
   string) {}
3   getValue(): T { return this.value; }
4   setColor(c: string) { this.color = c; }
5   getColor(): string | undefined { return this.color; }
6   \subsection{Visin general y contrato base}
7
8   Las restricciones (constraints) del sistema se modelan como clases que
   implementan contratos abstractos. Para mantener claridad se separan dos
   mbitos:
9
10  \begin{itemize}
11    \item Allocator (encargado de asignar franjas y das a las materias):
   restricciones de \textbf{da} y restricciones de \textbf{franja} (slot).
12    \item Student (encargado de verificar horarios por estudiante):
   restricciones aplicadas al horario final del estudiante.
13  \end{itemize}
14
15  Los contratos base son los siguientes:
16  \begin{lstlisting}[language=TypeScript,caption={Contratos base para
   restricciones de asignacin}]
17  export abstract class DayConstraint {
18    abstract name: string;
19    abstract isSatisfied(day: string, materia: MateriaInput, assigned:
   FranjaHoraria[], ctx: DayConstraintContext): boolean;
20  }
```



```

21
22     export abstract class SlotConstraint {
23         abstract name: string;
24         abstract isSatisfied(franja: FranjaHoraria, materia: MateriaInput,
25 assigned: FranjaHoraria[], strict: boolean): boolean;
26     }

```

Listing 4.1: Definición simplificada de Vertex<T>

Y para restricciones sobre horarios de estudiantes:

```

1     export abstract class ScheduleConstraint {
2         abstract name: string;
3         abstract isSatisfied(candidate: AsignacionResultado, confirmed:
4 AsignacionResultado[], grupoElegido: Map<string, string>): boolean;
5     }

```

Listing 4.2: Contrato base para restricciones de horarios estudiantiles

4.2.2 Listado de restricciones (resumen)

Para facilitar la lectura listamos las restricciones agrupadas por ámbito y tipo. Las referencias entre corchetes corresponden a los códigos usados en el repositorio.

Allocator - Day constraints

- NoRepeatedDay [R2]: Un mismo grupo no puede tener dos bloques el mismo día.
- NoConsecutiveDay [R3]: Materias de 3 créditos no pueden tener días consecutivos.
- MinDayDistance [R5]: Materias de 4 créditos requieren distancia mínima entre días.
- NoHeavyDaySaturation [R8]: Evitar saturar un día con materias pesadas del mismo semestre/carrera.

Allocator - Slot constraints

- NoRepeatedSlotNumber [R4]: Materias de 2 créditos no repiten número de franja.
- LabPrefersAfternoon [R7]: Laboratorios prefieren franjas de tarde (aplicable en pasada estricta).
- HeavyHourRange [R6]: Materias 3+ créditos solo entre horas permitidas (ej. 08:00-20:00).
- NoConsecutiveSlot [R9]: No permitir franjas numéricamente consecutivas dentro del mismo día.

Student constraints

- NoOverlap: Evitar solapamientos temporales entre bloques confirmados.
- NoGroupRepetition: Un mismo grupo no puede aparecer dos veces en el mismo día.
- ConsistentGroup: Si se eligió un grupo para una materia, aceptar sólo ese grupo.
- NoImmediateSedeChange: No cambiar de sede en bloques inmediatamente consecutivos.
- MaxSubjectsPerDay: Límite de materias distintas por día en el horario del estudiante.
- MaxFatigue: Límite de "fatiga"(distancia) entre bloques según un mapa de distancias.

4.2.3 Diagrama de clases (restricciones)

4.2.4 Uso de imágenes (rutas y URLs)

El informe referencia diagramas e imágenes que pueden servirse desde rutas relativas locales o URLs remotas. En este repositorio utilizamos rutas relativas (referenciando la carpeta `images/`) para asegurar reproducibilidad; si requieres servirlos desde URLs remotas, basta cambiar las referencias de forma sistemática. Ejemplo de URL alternativa:

```
\includegraphics[width=0.9\textwidth]{https://mi-servidor.org/diagrams/constraints-cla
```

4.2.5 Implementación: verificador (Allocator)

El verificador de allocator agrupa reglas de día y de franja y las evalúa antes de aceptar una asignación:

```
1   export class AllocatorConstraintChecker {
2     constructor(private readonly dayConstraints: DayConstraint[], private
      readonly slotConstraints: SlotConstraint[]) {}
3
4     checkDay(day: string, materia: MateriaInput, assigned: FranjaHoraria[],
      ctx: DayConstraintContext) {
5       for (const c of this.dayConstraints) {
6         if (!c.isSatisfied(day, materia, assigned, ctx)) return { passes:
          false, failedConstraint: c.name };
7       }
8       return { passes: true };
9     }
10
11    checkSlot(franja: FranjaHoraria, materia: MateriaInput, assigned:
      FranjaHoraria[], strict: boolean): boolean {
```

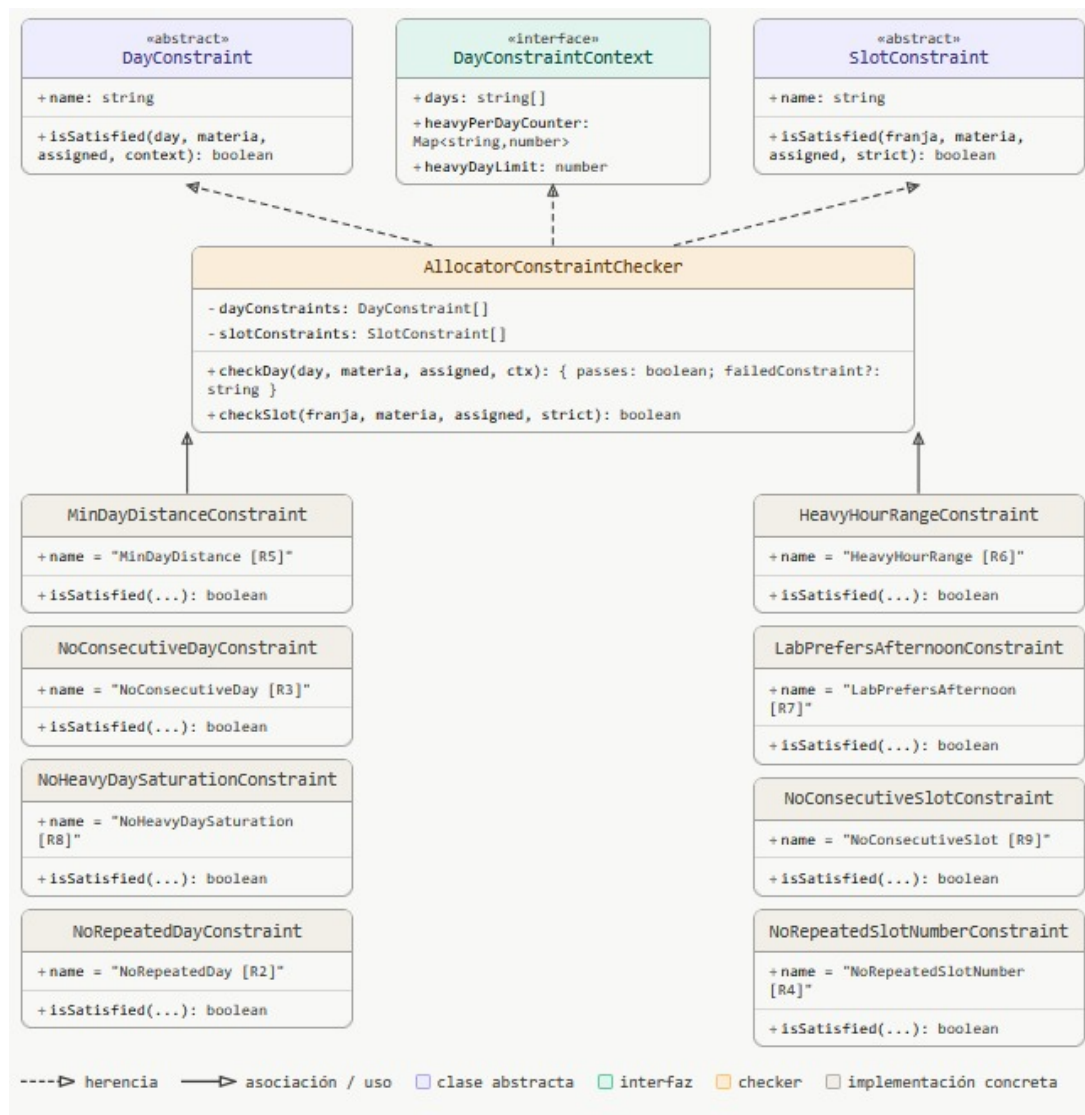


Figura 4.1: Diagrama de clases resumen de las restricciones (abstractas y concretas) — tanto para asignación (Allocator) como para horarios estudiantiles (Student)

```

12     return this.slotConstraints.every(c => c.isSatisfied(franja, materia,
13         assigned, strict));
14 }
15 }

```

Listing 4.3: Verificador de restricciones (Allocator)

4.2.6 Implementación: verificador (Student)

El verificador de horarios estudiantiles evalúa restricciones que operan sobre el conjunto de asignaciones confirmadas del estudiante:

```

1 export class ConstraintChecker {
2     constructor(private readonly constraints: ScheduleConstraint[]) {}

```

```

3
4     check(candidate: AsignacionResultado, confirmed: AsignacionResultado[],
5     grupoElegido: Map<string, string>) {
6         for (const constraint of this.constraints) {
7             if (!constraint.isSatisfied(candidate, confirmed, grupoElegido)) {
8                 return { passes: false, failedConstraint: constraint.name };
9             }
10        }
11        return { passes: true };
12    }
13

```

Listing 4.4: Verificador de restricciones (Student)

4.2.7 Ejemplos concretos

Un par de implementaciones concretas, extraídas del código base, para ilustrar el patrón:

```

1     export class LabPrefersAfternoonConstraint extends SlotConstraint {
2         name = "LabPrefersAfternoon [R7]";
3
4         isSatisfied(franja: FranjaHoraria, materia: MateriaInput, _assigned:
5         FranjaHoraria[], strict: boolean): boolean {
6             if (!strict || materia.tipo !== "laboratorio") return true;
7             const hora = parseInt(franja.inicio.split(":")[0] ?? "0", 10);
8             return hora >= 10;
9         }
10    }

```

Listing 4.5: Restricción concreta: preferencia de laboratorio por la tarde

```

1     export class NoOverlapConstraint extends ScheduleConstraint {
2         name = "NoOverlap";
3
4         isSatisfied(candidate: AsignacionResultado, confirmed:
5         AsignacionResultado[]): boolean {
6             return !confirmed.some(c => this.sesolapan(c, candidate));
7         }
8    }

```

Listing 4.6: Restricción concreta: no solapamiento (student)

4.2.8 Buenas prácticas y organización

Para que el informe sea legible y útil en su conjunto sugiero:

- Mantener la distinción entre `.allocator` "student.^{en} todos los ejemplos y diagramas.

- Documentar cada restricción con su propósito, ejemplos de entrada/salida y código mínimo (como se hizo arriba).
- Centralizar los diagramas en la carpeta `images/` y preferir rutas relativas para reproducibilidad; opcionalmente indicar URL alternativas.

Si quieres, puedo:

1. Cambiar todas las referencias de imágenes a URLs remotas.
2. Extraer los listados de restricciones a anexos individuales (uno por cada restricción) para mantener la sección principal más concisa.
3. Generar un índice detallado de clases/archivos del repositorio para que el lector pueda localizar la implementación exacta (por ejemplo: `src/universidad-proyecto/logic/schedule`).

Dime cuál de estas opciones prefieres y la aplico.

```
checkSlot(franja: FranjaHoraria, materia: MateriaInput, assigned: FranjaHoraria[], strict: boolean): boolean {
  return this.slotConstraints.every(c => c.isSatisfied(franja, materia, assigned, strict));
}
```

4.2.9 Ejemplo concreto: preferencia de laboratorio por la tarde

```
1 export class LabPrefersAfternoonConstraint extends SlotConstraint {
2   name = "LabPrefersAfternoon [R7]";
3
4   isSatisfied(franja: FranjaHoraria, materia: MateriaInput, _assigned:
5     FranjaHoraria[], strict: boolean): boolean {
6     if (!strict || materia.tipo !== "laboratorio") return true;
7     const hora = parseInt(franja.inicio.split(":")[0] ?? "0", 10);
8     return hora >= 10;
9   }
10 }
```

Listing 4.7: Restricción concreta de laboratorio

4.3 Constraints para horarios estudiantiles

Los horarios de estudiante usan un verificador separado. La estructura es equivalente: una clase abstracta, clases concretas y un chequeo central.

```
1 export abstract class ScheduleConstraint {
2   abstract name: string;
3   abstract isSatisfied(candidate: AsignacionResultado, confirmed:
4     AsignacionResultado[], grupoElegido: Map<string, string>): boolean;
5 }
```

```
6 export class ConstraintChecker {
7   constructor(private readonly constraints: ScheduleConstraint[]) {}
8
9   check(candidate: AsignacionResultado, confirmed: AsignacionResultado[],
10     grupoElegido: Map<string, string>) {
11     for (const constraint of this.constraints) {
12       if (!constraint.isSatisfied(candidate, confirmed, grupoElegido)) {
13         return { passes: false, failedConstraint: constraint.name };
14       }
15     }
16     return { passes: true };
17   }
18 }
```

Listing 4.8: Verificador de restricciones estudiantiles

4.3.1 Ejemplo concreto: no solapamiento

```
1 export class NoOverlapConstraint extends ScheduleConstraint {
2   name = "NoOverlap";
3
4   isSatisfied(candidate: AsignacionResultado, confirmed: AsignacionResultado
5     []): boolean {
6     return !confirmed.some(c => this.sesolapan(c, candidate));
7   }
8 }
```

Listing 4.9: Restricción concreta de no solapamiento

Con esta estructura, una nueva restricción se agrega creando una clase que implemente el contrato y registrándola en el verificador correspondiente.

5. Desglose de src/universidad-proyecto

5.1 Resumen funcional

La carpeta `src/universidad-proyecto` contiene la solución aplicada completa: cargadores de datos, construcción del grafo de conflictos, solucionadores, API y frontend. El flujo principal de ejecución es:

1. Carga y normalización de datos (`scheduler-data-loader.ts`).
2. Construcción del grafo de conflictos (`ConflictGraphBuilder`).
3. Separación por tipo de salón y ejecución de algoritmos de coloreo (`ScheduleSolver`).
4. Generación del horario semanal final (mapeo vértice→salón→franja).
5. Exposición por API (controladores y rutas) y visualización en frontend.

5.2 Punto de entrada

El archivo `index.ts` solo arranca la API:

```
1 import {SchedulerApiServer} from "../api/server";
2
3 const port = parseInt(process.env.PORT || "3002", 10);
4 const server = new SchedulerApiServer(port);
5 server.start();
```

Listing 5.1: Entrada principal del módulo universitario

5.3 Componentes clave

5.3.1 Cargador de datos

Normaliza JSON de entrada en estructuras tipadas (materias, salones, franjas) para el resto del pipeline.

5.3.2 ConflictGraphBuilder

Convierte materias y grupos en vértices y añade aristas cuando existe conflicto (misma franja, docente, semestre compartido, tipo incompatible).

5.3.3 ScheduleSolver

Recibe el grafo de conflictos y el listado de salones. Separa subgrafos por tipo de salón y aplica el algoritmo de coloreo seleccionado. Devuelve un mapeo de vértices a salones.

```
1 const input = SchedulerDataLoader.build().getData();
2 const { graph } = new ConflictGraphBuilder(input).build();
3 const algoritmo = new DSatur<GrupoData>(graph, input.salones.map(s => s.id));
4 const horario = new ScheduleBuilder(graph, input.salones).buildHorario(
    algoritmo); // aqui internamente llama a ScheduleSolver
```

Listing 5.2: Ejecución del solucionador global

5.3.4 ScheduleService

Singleton que mantiene en memoria el resultado del cálculo global para evitar recomputar en cada solicitud. Provee métodos para obtener `getHorarioGlobal()` y `getGrafoInfo()`.

```
1 async getHorarioGlobal(): Promise<HorarioSemanal> {
2   if (this.horarioGlobal) return this.horarioGlobal;
3
4   const input = SchedulerDataLoader.build().getData();
5   const { graph } = new ConflictGraphBuilder(input).build();
6   const algoritmo = new DSatur<GrupoData>(graph, input.salones.map(s => s.id)
7     );
8   const horario = new ScheduleBuilder(graph, input.salones).buildHorario(
9     algoritmo);
10
11   this.horarioGlobal = horario;
12   return horario;
13 }
```

Listing 5.3: Servicio global en memoria

5.3.5 StudentScheduleSolver

Genera horarios personalizados para un estudiante a partir del horario global y los grupos elegidos. Integra Dijkstra y backtracking según disponibilidad.

```
1 async createSchedule(horario: HorarioSemanal, salones: SalonInput[], request:
    StudentScheduleRequest) {
2   const solver = new StudentScheduleSolver(request);
3   const resultado = solver.execute();
4   this.horariosEstudiantes.set(request.estudianteId, resultado);
```



```
5  return resultado;  
6  }
```

Listing 5.4: Creación del horario estudiantil

5.4 Rutas y controladores principales

Rutas expuestas:

- GET /schedule
- GET /schedule/grafó
- POST /student/schedule
- GET /student/schedule/:estudianteId
- GET /student/schedules

5.5 Consideraciones de diseño

- Separación clara entre construcción del grafo (lógica) y la API (exposición).
- Caché en memoria para mejorar latencia en consultas repetidas.
- Modularidad para poder reemplazar algoritmos de coloreo sin tocar la capa de datos.

6. Especificación técnica

6.1 Problema seleccionado

Asignación de salones y franjas horarias a bloques académicos. Cada bloque necesita un salón compatible con su tipo y no puede coincidir con otros bloques que compartan estudiantes, profesor o franja horaria.

6.2 Cómo se modela en el código

El proyecto representa cada bloque como un vértice y cada conflicto como una arista. Los salones disponibles se usan como identificadores de asignación. El constructor del grafo recibe materias, grupos y salones, y devuelve la estructura que después consume el solucionador de horarios.

La implementación que genera el horario global se apoya en tres piezas:

- el cargador de datos, que normaliza el JSON de entrada;
- el constructor del grafo de conflictos, que detecta incompatibilidades;
- el solucionador global, que asigna salones y produce el horario semanal.

Podemos observarlo de la siguiente forma:

```
1 export class VertexFactory {
2   static create(
3     materia: MateriaInput,
4     grupo: number,
5     bloque: number,
6     franja: FranjaHoraria
7   ): Vertex<GrupoData> {
8     const data: GrupoData = {
9       materiaId: materia.id,
10      nombre: `${materia.nombre} G${grupo}`,
11      grupo: String(grupo),
12      semestre: materia.semestre,
13      tipoSalon: materia.tipo,
14      carrera: materia.carrera,
15      creditos: materia.creditos,
```

```
16         franja: franja,  
17         salon: null,  
18         bloque: bloque,  
19     };  
20  
21     const vertexId = `${materia.id}-G${grupo}-B${bloque}`;  
22     return new Vertex<GrupoData>(vertexId, data);  
23 }  
24 }  
25  
26 // ... en el archivo de conflict-graph.builder.ts  
27  
28 private createConflictEdges(): void {  
29     EdgeCreator.connectByFanja(this.vertices, this.graph);  
30     EdgeCreator.connectSameGroupSameDay(this.vertices, this.graph);  
31 } // aqui vemos que usa edge paracrear los conflictos y luego  
solucionarlos
```

Listing 6.1: Modelado del negocio

6.3 Algoritmos seleccionados

El proyecto usa dos familias de algoritmos en la ruta principal:

- coloreo de grafos para la asignación global de salones;
- Dijkstra para construir horarios personalizados de estudiantes cuando se necesitan rutas de menor costo entre clases.

6.4 Entrada y salida

Entrada: archivos JSON con materias, salones y franjas. Salida: horario semanal global y horarios individuales para estudiantes, ambos en estructuras JSON/TypeScript que pueden consumirse desde la API.

6.5 Validación básica

Con los datos incluidos en el repositorio se puede generar un horario global y recuperar la información del grafo. Para horarios estudiantiles, la validación consiste en revisar que el conjunto de materias solicitado tenga una asignación compatible y que no exista solapamiento entre bloques.

7. API y endpoints

7.1 Visión general

La API expone el horario global, la información del grafo y los horarios personalizados de estudiantes. Todas las respuestas usan JSON y la capa de servicio mantiene en memoria el horario global ya calculado.

7.2 GET /schedule

Request:

```
1 GET /schedule
```

Response:

```
1 {
2   "success": true,
3   "data": {
4     "horario": [
5       {
6         "grupo": "SIS-ALG-G1",
7         "materia": "lgebra",
8         "semestre": 1,
9         "franja": "Lun-A1",
10        "horario": "Lunes 08:00-10:00",
11        "salon": "A101",
12        "salonDetalle": {
13          "id": "A101",
14          "tipo": "normal",
15          "capacidad": 40,
16          "piso": 1,
17          "sede": "Principal"
18        },
19        "tipoSalon": "normal",
20        "capacidad": 40,
21        "color": "A101"
22      }
23    ],
24    "grafo": {
```

```
25     "vertices": 108,  
26     "aristas": 1243  
27   }  
28 }  
29 }
```

Listing 7.1: Respuesta de /schedule

7.3 GET /schedule/grafos

Request:

```
1 GET /schedule/grafos
```

Response:

```
1 {  
2   "success": true,  
3   "data": {  
4     "vertices": 108,  
5     "aristas": 1243  
6   }  
7 }
```

Listing 7.2: Respuesta de /schedule/grafos

7.4 POST /student/schedule

Request:

```
1 {  
2   "estudianteId": "EST-20221020084",  
3   "materias": ["SIS-ALG", "SIS-BD1", "SIS-PROG"]  
4 }
```

Listing 7.3: Body de /student/schedule

Response:

```
1 {  
2   "success": true,  
3   "data": {  
4     "estudianteId": "EST-20221020084",  
5     "bloques": [  
6       {  
7         "materiaId": "SIS-ALG",  
8         "nombreMateria": "lgebra",  
9         "grupo": "SIS-ALG-G1",  
10        "dia": "Lunes",
```

```
11     "horaInicio": "08:00",
12     "horaFin": "10:00",
13     "salon": "A101",
14     "sede": "Principal",
15     "piso": 1,
16     "tipoSalon": "normal",
17     "bloque": 1
18   }
19 ],
20   "metricas": {
21     "totalBloques": 3,
22     "cambiosDeSede": 1,
23     "costoTotal": 4,
24     "sedesUsadas": ["Principal"],
25     "materiasNoAsignadas": []
26   }
27 }
28 }
```

Listing 7.4: Respuesta de /student/schedule

7.5 GET /student/schedule/:estudianteId

Request:

```
1 GET /student/schedule/EST-20221020084
```

Response:

```
1 {
2   "success": true,
3   "data": {
4     "estudianteId": "EST-20221020084",
5     "bloques": [/* ... */],
6     "metricas": { /* ... */ }
7   }
8 }
```

Listing 7.5: Respuesta de /student/schedule/:estudianteId

7.6 GET /student/schedules

Request:

```
1 GET /student/schedules
```

Response:

```
1 {
2   "success": true,
3   "data": {
4     "total": 1,
5     "estudiantes": [
6       {
7         "estudianteId": "EST-20221020084",
8         "totalBloques": 3,
9         "costoTotal": 4,
10        "cambiosDeSede": 1,
11        "sedesUsadas": ["Principal"],
12        "materiasNoAsignadas": []
13      }
14    ]
15  }
16 }
```

Listing 7.6: Respuesta de /student/schedules

7.7 Formato de respuesta

Todas las rutas siguen la estructura:

```
1 interface ApiResponse<T> { success: boolean; data?: T; error?: string }
```

8. Modelado del problema

8.1 Descripción del conflicto

El sistema trabaja con bloques académicos que deben ubicarse en salones y franjas horarias. Un bloque no puede compartir franja con otro bloque que use los mismos recursos o que implique conflicto de estudiantes, profesor o tipo de salón.

8.2 Interpretación del grafo

Cada bloque se convierte en un nodo del grafo. Cuando dos bloques no pueden convivir en el mismo horario o en el mismo salón, se agrega una conexión de conflicto entre ellos. Los salones disponibles se usan como etiquetas de asignación para decidir dónde queda cada bloque.

8.3 Reglas que aplica el código

- materias tipo laboratorio solo pueden ir a salones de laboratorio;
- materias normales solo pueden ir a salones normales;
- bloques con choque de horario se consideran incompatibles;
- los bloques personalizados de estudiante no pueden superponerse;
- el sistema mantiene en memoria el horario ya calculado para evitar recomputaciones innecesarias.

8.4 Resultado esperado

El resultado es un horario válido que asigna un salón a cada bloque y permite recuperar tanto el horario global como el horario por estudiante desde la API.

9. Algoritmos empleados

9.1 Ruta principal del proyecto

La solución usa dos algoritmos en la ruta principal:

- **Coloreo de grafos:** se usa para asignar salones al horario global. El solucionador global recibe el grafo de conflictos y produce una asignación compatible con el tipo de salón.
- **Dijkstra:** se usa en la planificación estudiantil para construir un horario personalizado cuando se necesita minimizar el costo de desplazamiento y la fatiga.

9.2 Coloreo global

El horario global se construye con el solucionador que opera sobre el grafo de conflictos. En el código se utiliza D-Satur como estrategia principal dentro de la capa de servicio global.

9.3 Planificación estudiantil

El horario de estudiante se genera a partir del horario global y del conjunto de materias solicitado. La estructura de costos se resuelve con un solver que puede encadenar estrategias, pero la lógica central documentada aquí es la búsqueda de menor costo sobre las alternativas disponibles.

9.4 Otras heurísticas presentes en el repositorio

También existen implementaciones de Welsh-Powell y coloreo voraz en la carpeta de algoritmos. Permanecen en el repositorio como alternativas de código, pero esta documentación se concentra en la ruta que usa el sistema al generar el horario.

10. Ejecución de algoritmos

10.1 Construcción del horario global

El horario global se construye a partir de los datos cargados, el grafo de conflictos y el solucionador principal.

```
1 const input = SchedulerDataLoader.build().getData();
2 const { graph } = new ConflictGraphBuilder(input).build();
3 const algoritmo = new DSatur<GrupoData>(graph, input.salones.map(s => s.id));
4 const horario = new ScheduleBuilder(graph, input.salones).buildHorario(
    algoritmo);
```

Listing 10.1: Flujo principal del horario global

10.2 Uso de constraints durante la asignación

Antes de aceptar una franja o un bloque, el sistema consulta los verificadores de restricciones. Cada restricción concreta extiende una clase abstracta y devuelve si la asignación es válida.

```
1 const checker = new AllocatorConstraintChecker(dayConstraints,
    slotConstraints);
2 const dayResult = checker.checkDay(dia, materia, asignadas, contexto);
3 const slotResult = checker.checkSlot(franja, materia, asignadas, true);
```

Listing 10.2: Consulta de restricciones

10.3 Planificación estudiantil

Cuando llega una solicitud de estudiante, el controlador valida el body, carga el horario global y delega la construcción al servicio de estudiantes.

```
1 async createSchedule(req: Request, res: Response): Promise<void> {
2   const body = req.body;
3   if (!body.estudianteId || !Array.isArray(body.materias) || body.materias.
       length === 0) {
4     res.status(400).json({success: false, error: "Se requiere estudianteId y
       materias[]."});
5   }
```

```
5     return;
6   }
7
8   const input = SchedulerDataLoader.build().getData();
9   const horario = await scheduleService.getHorarioGlobal();
10  const resultado = await studentService.createSchedule(horario, input.
    salones, body);
11  res.json({success: true, data: resultado});
12 }
```

Listing 10.3: Controlador de horarios de estudiante

10.4 Recuperación de datos ya calculados

El servicio de horario global mantiene caché en memoria y el servicio de estudiantes guarda el resultado por ID. Eso permite devolver el horario sin recalculiar todo el grafo en cada solicitud.

```
1  async getHorarioGlobal(): Promise<HorarioSemanal> {
2    if (this.horarioGlobal) return this.horarioGlobal;
3    const input = SchedulerDataLoader.build().getData();
4    const { graph } = new ConflictGraphBuilder(input).build();
5    const algoritmo = new DSatur<GrupoData>(graph, input.salones.map(s => s.id)
      );
6    const horario = new ScheduleBuilder(graph, input.salones).buildHorario(
      algoritmo);
7    this.horarioGlobal = horario;
8    return horario;
9  }
```

Listing 10.4: Caché de horario global

11. Interfaz de usuario y visualización

11.1 Arquitectura del frontend

La interfaz es una SPA React que consume la API para obtener el horario global y generar horarios estudiantiles. Componentes clave: `ScheduleApp`, `SchedulePage`, `StudentsPage`. Hooks: `useSchedule`, `useStudents`.

11.2 Placeholders para capturas

Colocar las imágenes en la carpeta `figuras/` con los nombres exactos:

1. `frontend-horario-global.png`: vista de la matriz días×franjeas.
2. `frontend-formulario-estudiante.png`: formulario de selección de grupos.
3. `frontend-horario-estudiante.png`: horario generado para estudiante.
4. `flujo-algoritmos.png`: diagrama del flujo ejecución (datos→grafo→coloreo→horario; rama estudiantil con Dijkstra).

11.3 Ejemplo de hook (resumen)

```
1 export const useSchedule = () => {
2   const [horario, setHorario] = useState([]);
3   useEffect(() => {
4     fetch('/schedule').then(r => r.json()).then(j => setHorario(j.data.
      horario));
5   }, []);
6   return { horario };
7 };
```

Listing 11.1: Hook `useSchedule` (resumen)

11.4 Flujo de interacción

Usuario selecciona pestaña "Horario Global."° "Horarios Estudiantiles"; para generar un horario estudiantil se envía un POST a `/student/schedule` y la respuesta se renderiza en la interfaz con métricas.

12. Conclusiones

La solución presentada provee un pipeline reproducible para asignación de salones y generación de horarios basado en grafos y heurísticas. Los hallazgos principales son:

- la combinación de coloreo global y Dijkstra para casos estudiantiles permite cubrir tanto la asignación masiva como necesidades individuales;
- D-Satur produce coloraciones compactas pero con mayor costo computacional; Welsh-Powell y voraz son alternativas útiles para comparación y reducción de tiempo;
- la arquitectura separada facilita evaluar y reemplazar heurísticas sin afectar la API ni el frontend.

Tareas pendientes para completar entregables: añadir el escenario simulado, recolectar métricas en los 3 escenarios y subir capturas del frontend.

13. Anexos

13.1 Estructura del repositorio

Carpetas relevantes:

- `src/algoritmos`
- `src/estructuras`
- `src/universidad-proyecto` (datos, lógica, API, frontend)
- `src/docs/sections` (archivos `.tex` de esta documentación)

13.2 Datasets

Archivos incluidos: `src/universidad-proyecto/data/data.json` y `data2.json`. Uno de ellos se usa como instancia principal para la validación y el otro queda como entrada alternativa.

13.3 Imágenes

Nombres esperados en `figuras/`:

- `frontend-horario-global.png`
- `frontend-formulario-estudiante.png`
- `frontend-horario-estudiante.png`
- `flujo-algoritmos.png`

13.4 Comandos útiles

Ejemplo de arranque del servidor:

```
npx ts-node src/universidad-proyecto/index.ts
```